Evidencias de pruebas — Linked List

Pruebas ejecutadas sobre el código fuente incluido en el proyecto. No son resultados simulados: se generaron al compilar y ejecutar app y tests_runner.

53/53	0	12/12
Pruebas automáticas aprobadas	Fallos reportados	Opciones del menú verificadas

Matriz de evidencias: 12 operaciones obligatorias

Opción	Operación	Entrada de prueba	Salida observada
1	addFirst	5	[5, 10, 20, 30]
2	addLast	40	[5, 10, 20, 30, 40]
3	insert	índice 1, dato 15	[5, 10, 15, 20, 30, 40]
4	deleteData	20	[5, 10, 15, 30, 40]
5	deleteAt	índice 0	[10, 15, 30, 40]
6	getData	índice 2	30
7	updateData	15 → 16	[10, 16, 30, 40]
8	updateAt	índice 1 → 17	[10, 17, 30, 40]
9	findData	30	2
10	operator[ ] leer	índice 0	10
11	operator[ ] escribir	índice 3 → 45	[10, 17, 30, 45]
12	operator=	duplicar	Copia: [10, 17, 30, 45]

Nota: el método insert(i, dato) coloca el nodo en el índice i. La opción 3 del menú inserta después del índice solicitado usando insert(i + 1, dato). Todas las posiciones empiezan en cero.

Registro real del menú — enteros

Ejecución reproducible: ./app < tests/menu_int_input.txt. Lista inicial capturada: [10 -> 20 -> 30]. Se registran solo las líneas de resultados; el archivo tests/menu_int_output.txt incluye el menú completo.

Registro 1:
```
Selecciona: Elemento 0: Elemento 1: Elemento 2: Lista creada: [10 -> 20 -> 30]
```

Registro 2:
```
Opcion: Nuevo dato: Agregado al principio.
```

Registro 3:
```
Opcion: Nuevo dato: Agregado al final.
```

Registro 4:
```
Opcion: Indice DESPUES del cual insertar: Nuevo dato: Insertado despues del indice 1.
```

Registro 5:
```
Opcion: Dato a borrar: Elemento eliminado.
```

Registro 6:
```
Opcion: Indice a borrar: Elemento eliminado.
```

Registro 7:
```
Opcion: Indice a consultar: getData(2) = 30
```

Registro 8:
```
Opcion: Valor a buscar: Valor nuevo: Elemento actualizado.
```

Registro 9:
```
Opcion: Indice a modificar: Valor nuevo: Elemento actualizado.
```

Registro 10:
```
Opcion: Valor a buscar: findData = 2
```

Registro 11:
```
Opcion: Indice a consultar con []: lista[0] = 10
```

Registro 12:
```
Opcion: Indice a modificar con []: Valor nuevo: Elemento actualizado con [].
```

Registro 13:
```
Opcion: Lista duplicada con operador =. Copia: [10 -> 17 -> 30 -> 45]
```

Comprobación adicional: copia independiente

Después de duplicar la lista, se agregó 99 a la lista original y se imprimió nuevamente la copia. Los nodos de la copia conservaron sus valores.

```
Lista actual: [99 -> 10 -> 17 -> 30 -> 45]
```

```
Copia guardada: [10 -> 17 -> 30 -> 45]
```

Pruebas automáticas — salida completa

Comando: ./tests_runner. Estas pruebas incluyen las 12 operaciones, listas vacías, índices inválidos, elementos repetidos, listas de int y string, autoasignación y copia profunda.

```
[PASS] 01 Lista inicialmente vacia
[PASS] 02 Tamano inicialmente cero
[PASS] 03 deleteData en lista vacia retorna false
[PASS] 04 deleteAt en lista vacia retorna false
[PASS] 05 findData en lista vacia retorna -1
[PASS] 06 getData invalido lanza out_of_range
[PASS] 07 operator[] invalido lanza out_of_range
[PASS] 08 addFirst agrega al inicio
[PASS] 09 addFirst incrementa size
[PASS] 10 addLast agrega al final
[PASS] 11 addLast incrementa size
[PASS] 12 insert en posicion 0
[PASS] 13 insert intermedio
[PASS] 14 insert al final
[PASS] 15 insert negativo lanza out_of_range
[PASS] 16 insert mayor a size lanza out_of_range
[PASS] 17 getData regresa posicion correcta
[PASS] 18 updateData reemplaza primera coincidencia
[PASS] 19 updateData ausente lanza out_of_range
[PASS] 20 updateAt reemplaza por indice
[PASS] 21 updateAt fuera de rango lanza
[PASS] 22 findData encuentra indice
[PASS] 23 findData ausente retorna -1
[PASS] 24 deleteData borra coincidencia
[PASS] 25 deleteData sin coincidencia false
[PASS] 26 deleteAt borra cabeza
[PASS] 27 deleteAt borra ultimo
[PASS] 28 deleteAt invalido retorna false
[PASS] 29 operator[] permite leer
[PASS] 30 operator[] permite actualizar
[PASS] 31 operator[] const permite leer
[PASS] 32 operator[] negativo lanza
[PASS] 33 operator= copia el contenido
[PASS] 34 Copia independiente del original
[PASS] 35 Original independiente de la copia
[PASS] 36 Autoasignacion segura
[PASS] 37 Constructor de copia es profundo
[PASS] 38 clear libera y deja lista vacia
[PASS] 39 Alias push_front funciona
[PASS] 40 Alias push_back funciona
[PASS] 41 Template string: addFirst
[PASS] 42 Template string: addLast
[PASS] 43 Template string: getData
[PASS] 44 Template string: findData
[PASS] 45 Template string: updateData
[PASS] 46 Template string: deleteData
[PASS] 47 Template string: copia profunda
[PASS] 48 print muestra flechas
[PASS] 49 deleteData elimina solo primera repeticion
[PASS] 50 deleteAt elimina ultimo nodo
[PASS] 51 deleteAt elimina unico nodo
[PASS] 52 getData negativo lanza
[PASS] 53 operator[] const fuera de rango lanza
```

Resultado comprobado: 53 aprobadas, 0 fallidas.

Tipos de datos, aleatorios y errores

Cadenas capturadas

```
Selecciona: Elemento 0: Elemento 1: Lista creada: [hola mundo -> buenos dias]
Opcion: Indice a consultar: getData(2) = hola mundo
Opcion: Valor a buscar: findData = 3
Opcion: Indice a consultar con []: lista[0] = nuevo
Opcion: Copia guardada: [nuevo -> cambiado -> hola mundo -> buenos dias -> al final]
```

Creación de datos aleatorios

```
Selecciona: Lista creada: [cielo -> nube -> rio -> nube -> mar]
Selecciona: Lista creada: [72 -> 91 -> 0 -> 87 -> 97]
```

Control de errores y posiciones inválidas

```
Opcion: Indice a consultar: getData(-1) = Error de rango: Indice fuera de rango
Opcion: Valor a buscar: Valor nuevo: Error de rango: Dato no encontrado para actualizar
Opcion: Indice a borrar: Indice invalido.
```

Pruebas independientes

El programa test_node demuestra que dos nodos pueden conectarse mediante next:

```
node2 data: 10
node2->next data: 20
node1 data: 20
```

El programa pointer también compila y demuestra manejo de apuntadores y unique_ptr sin acceder a memoria liberada.

Para repetir las pruebas, consulta README.md y los archivos de tests/. Las cifras aleatorias cambian en cada nueva ejecución.